nixlab – Projektdokumentation

Stand: 19.10.2026

> Diese Dokumentation beschreibt ein privates Repo. Domain und interne Adressen sind durch Beispielwerte ersetzt: die Domain durch `homelab.example` (RFC 2606), IPv4 nach RFC 5737 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`), IPv6 nach RFC 3849 (`2001:db8::/32`), der Tailnet-Name durch `example-tailnet.ts.net`. Die echte Mesh-Range ist ein /22 aus RFC 1918. Module, Dokumentation und Build-Werkzeuge sind zusammen mit Claude (Anthropic) entstanden. Lizenz: MIT.

Überblick

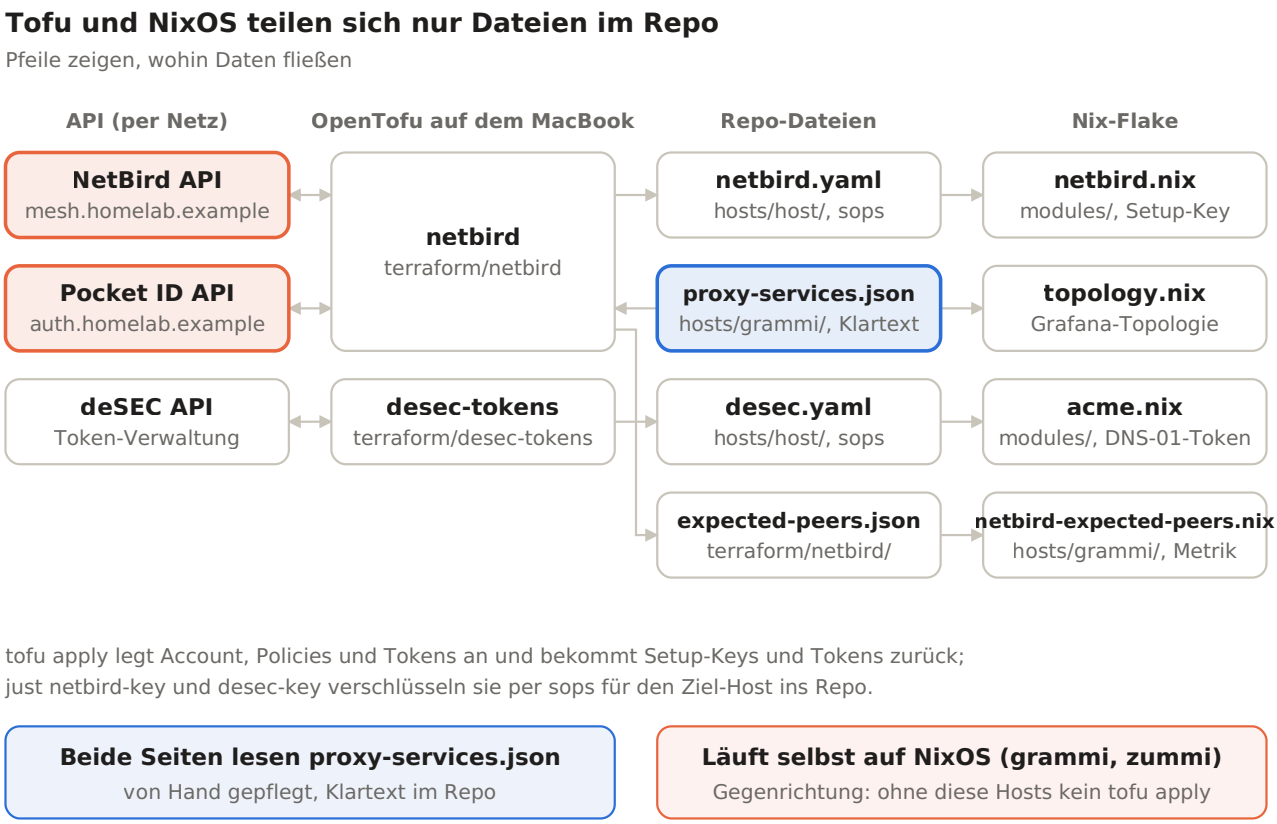
nixlab ist das Flake-Monorepo für das Homelab **homelab.example**: fünf NixOS-Hosts und ein Mac mini mit nix-darwin, dazu drei OpenTofu-Module für den NetBird-Account, die deSEC-API-Tokens und die GitHub-Repos samt Deploy-Keys. Bedient wird alles über ein `justfile` vom MacBook aus; die Hosts ziehen ihre Konfiguration selbst aus dem privaten GitHub-Repo und aktualisieren sich nachts.

Kurzantwort auf die Frage *„Hast du NixOS und OpenTofu irgendwie miteinander verdrahtet?“*: **Ja, aber bewusst lose – über Dateien im Repo, nicht über ein gemeinsames Tool.** OpenTofu schreibt sops-verschlüsselte Dateien in `hosts/<host>/`, die NixOS-Module beim Build lesen, und beide Seiten lesen dieselbe Dienstliste `hosts/grammi/proxy-services.json`. Kein terranix, kein Tofu-Lauf aus Nix heraus. Details unter NixOS & OpenTofu.

Zielgruppe: Leute, die NixOS-Flakes, sops-nix und Terraform/OpenTofu lesen können.

NixOS & OpenTofu

Ja, beides ist verdrahtet – über das Repo als gemeinsame Schnittstelle. OpenTofu verwaltet das, was außerhalb der Maschinen per API existiert (NetBird-Account, deSEC-Tokens, Pocket-ID-Gruppen); NixOS verwaltet die Maschinen. Die beiden reden nie direkt miteinander: Tofu läuft nur auf dem MacBook über `just`, Nix wertet nie Tofu-State aus.



Tofu schreibt nie direkt auf einen Host: jedes Ergebnis landet als Datei im Repo und erreicht den Host erst mit dem nächsten Deploy.

Die Berührungspunkte

Richtung	Datei	Erzeugt von	Gelesen von	Inhalt
Tofu → NixOS	`hosts/<host>/netbird.yaml`	`just netbird-key <host>` (`tofu output setup_keys` → `sops encrypt`)	`modules/netbird.nix` (`sopsFile`)	Einmal-Setup-Key, 24 h gültig
Tofu → NixOS	`hosts/<host>/desec.yaml`	`just desec-key <host>` (`tofu output acme_tokens` → `sops encrypt`)	`modules/acme.nix` (`sopsFile`)	deSEC-Token nur für TXT-Records, VMs auf ihre IP gepinnt
Tofu → NixOS	`terraform/netbird/expected-peers.json`	`just netbird-apply` (`tofu output expected_peers`, aus den Policies in `expected-peers.tf` berechnet)	`hosts/grammi/netbird-expected-peers.nix` (`builtins.readFile`)	Soll-Anzahl NetBird-Peers je Host als Metrik `netbird_expected_peers`, im NetBird-Dashboard neben der echten Network Map (z. B. aldrin 6, grammi 8)

Richtung	Datei	Erzeugt von	Gelesen von	Inhalt
Tofu → NixOS	`terraform/netbird/admin-peers.json`	`just netbird-apply` (`tofu output admin_peers`, aus der Gruppe `admin-workstations`, `admin-peers.tf`)	`flake.nix` (`builtins.readFile`, Option `services.docsSite.allow`)	Mesh-Adressen der Admin-Geräte – wer die private Doku lesen darf
gemeinsam	`hosts/grammi/proxy-services.json`	von Hand	`terraform/netbird/reverse-proxy.tf` (`jsondecode(file(...))`) und `topology.nix`	Name → App, Port, Gruppen der Reverse-Proxy-Dienste
Repo → beide	`hosts/<host>/github.yaml` + `github-deploy.pub`	`just github-key <host>` (`ssh-keygen` → `sops encrypt`)	privat: `modules/deploy.nix` (`sopsFile`); öffentlich: `terraform/github` (`file(...)`)	Deploy-Key für `/etc/nixos`; Tofu sieht nur den öffentlichen Teil
NixOS → Tofu	NetBird-Server auf grammi	`hosts/grammi/netbird.nix`	Provider über `https://mesh.homelab.example`	Die API, gegen die Tofu überhaupt erst läuft
NixOS → Tofu	Bootstrap-Tokens	`just netbird-setup-owner`, `netbird-proxy-token` (SSH auf grammi, Server-API)	`terraform/netbird/secrets.yaml` bzw. `hosts/grammi/netbird-proxy.yaml`	Owner-PAT, Proxy-Token

Der Ablauf für einen neuen Host: Tofu legt den Setup-Key an, `just netbird-key` schreibt ihn verschlüsselt für genau diesen Host (age-Key aus seinem SSH-Host-Key) nach `hosts/<host>/netbird.yaml`, Commit, Deploy innerhalb von 24 h. Der Host registriert sich beim Start selbst; Peers selbst verwaltet Tofu nicht, es findet sie per `data "netbird_peer"` über den Namen.

Was bewusst nicht verdrahtet ist

- **Keine gemeinsame Host-Liste.** Hosts stehen dreimal: `nixosConfigurations` in `flake.nix`, `local.hosts` in den Tofu-Modulen und `hosts :=` im `justfile`. Ein neuer Host muss an allen drei Stellen eingetragen werden.
- **Kein terranix / kein Tofu im Nix-Build.** Tofu-State liegt lokal (gitignored, im Haupt-Checkout, auch aus Worktrees), weil er Setup-Keys und Tokens im Klartext hält.
- **Kein Provisioning der Maschinen.** Die VMs werden beim Anbieter von Hand angelegt, ursalia/cuckoosegg nach `docs/install-*.md`.
- **Öffentliches DNS** liegt in DNSControl (eigenes Repo, nur per Symlink eingebunden), nicht in Tofu.

Die Grenze ist also: Was Nix aus dem Repo heraus deterministisch bauen kann, macht Nix. Was nur einmal per API erzeugt wird und ein Geheimnis zurückgibt, macht Tofu – und übergibt es verschlüsselt als Datei.

Secrets, Netzwerk & Zugriff

Secrets (sops-nix, age)

Jede Host-Datei ist für genau zwei Empfänger verschlüsselt: den age-Key aus dem SSH-Host-Key des Hosts und den Offline-Recovery-Key `admin`. Das MacBook kann Host-Secrets also nicht lesen – `just`

`secrets <host>` editiert sie auf dem Host und holt das Ergebnis zurück. Verschlüsseln für einen Host geht dagegen überall (nur der öffentliche Key ist nötig); genau das nutzen `netbird-key` und `desec-key`.

Datei	Empfänger	Inhalt
`hosts/<host>/secrets.yaml`	Host + admin	Deploy-Key, App-Passwörter, OIDC-Clients, restic
`hosts/<host>/netbird.yaml`	Host + admin	NetBird-Setup-Key (von Tofu)
`hosts/<host>/desec.yaml`	Host + admin	deSEC-Token (von Tofu)
`hosts/<host>/github.yaml`	Host + admin	GitHub-Deploy-Key (von `just github-key`)
`terraform/*/secrets.yaml`	workstation + admin	API-Tokens für die Provider

Der Workstation-Key liegt in 1Password und wird über `SOPS_AGE_KEY_CMD` gelesen, nie als Datei. `env.sh` in jedem Tofu-Modul entschlüsselt die Tokens nur in die Shell des jeweiligen Rezepts.

Netzwerk

- **NetBird-Mesh** (`192.0.2.0/24`, Peer-Namen unter `peer.homelab.example`): Control Plane auf grammi, Login über Pocket ID. Private Range (RFC 1918) statt CGNAT, weil tailscaled Pakete aus 100.64/10 verwirft, solange beide Clients parallel laufen.
- **Gruppen:** `servers` (alle Hosts außer cuckoosegg), `subraum-devices`, `admin-workstations` (MacBook), dazu Personengruppen aus Pocket ID.
- **Heim-LAN** (`198.51.100.0/24`): Route über ursalia (Metrik 100), die beiden Apple TVs als Rückfall (Gruppe `home-routers`, Metrik 200, eigener Setup-Key per `just netbird-router-key`, keiner Person zugeordnet, in keiner Policy); für Gruppe `home-lan`, nicht für Geräte, die schon zu Hause sind (Posture-Check `not-at-home`). Die Apple TVs brauchen in der App „Quantum-Resistance“, sonst nimmt kein Peer (Rosenpass strikt) einen Tunnel zu ihnen an.
- **Anbindung von ursalia:** WLAN ins Hauptnetz des Heim-LANs (Intel BE200, `hosts/ursalia/wifi.nix`): wpa_supplicant, WPA3 mit WPA2-Rückfall, Land DE, alle Bänder, Passwort per sops. Am endgültigen Platz gibt es kein Kabel; eno1 behält DHCP als Rückfall mit schlechterer Routen-Metrik, damit das WLAN die Hauptverbindung bleibt, wenn beide aktiv sind.
- **Anbindung von zorlock:** ebenfalls WLAN (`en1`, Broadcom); `hosts/zorlock/wifi.nix` stellt sicher, dass `homelab` als bevorzugtes Netz eingetragen ist (`networksetup`, fügt nur hinzu, Passwort per sops in `hosts/zorlock/wifi.yaml`). nix-darwin hat kein WLAN-Modul; die Bandwahl bleibt bei macOS.
- **Subraum:** Route über cuckoosegg für `subraum-access`. cuckoosegg selbst darf nur grammis Ingest (TCP 8428/9428) erreichen und nur vom MacBook per SSH erreicht werden.
- **Tailscale** läuft während der Migration noch parallel und dient als Break-Glass: `NIXLAB_DOMAIN=example-tailnet.ts.net just deploy zummi`.

Zugriff

Öffentlich offen sind nur 22/80/443. root-SSH lässt `modules/ssh.nix` nur von Mesh- oder Tailscale-Adressen zu; jeder Host hat ein eigenes Login-Keypair. Das Mesh ist derzeit nur IPv4, Dual-Stack ist als offener Punkt notiert.

Private Doku: nur Admin-Geräte

Die private Fassung dieser Doku (`doku.homelab.example`, auf aldrin und ursalia) enthält jede interne Adresse und beschreibt, wie das Lab betrieben wird. Das ist mehr, als jedes Gerät im Mesh sehen muss. Deshalb liefert nginx sie nur an die Mesh-Adressen der Admin-Geräte aus; alle anderen bekommen 403 – mit einer eigenen Seite, die sagt, warum, von welcher Adresse die Anfrage kam und wie man ein Gerät freischaltet (ohne Navigation, damit die Seitentitel nicht sichtbar werden). Der Name selbst löst für alle Peers auf.

Von außen führt derselbe Name auf die öffentliche, anonymisierte Doku (GitHub Pages): Öffentlich ist `doku` ein CNAME auf den Reverse Proxy, dessen Dienst `doku` ohne Login an einen Listener auf aldrin (Port 8091, nur Mesh) weiterreicht, der nur eine Weiterleitung kennt. Wer im Mesh ist, kommt über die NetBird-Zone direkt zu aldrin und ursalia.

Wer	Weg	Ergebnis
Extern oder NetBird aus	Reverse Proxy (grammi) → aldrin:8091	301 auf die öffentliche Doku
Im Mesh, Gerät in `admin-workstations`	direkt aldrin/ursalia	private Doku
Im Mesh, anderes Gerät	direkt aldrin/ursalia	403-Seite

Warum über den Proxy: So steht `doku` mit den anderen öffentlichen Namen im NetBird-Dashboard, statt als eigener öffentlicher vHost auf aldrin. Der Proxy selbst kann nicht umleiten, nur weiterreichen – daher der Weiterleitungs-Listener. Er hat einen eigenen Port, statt dass der Doku-vHost den Proxy an seiner Adresse erkennt: Der Proxy ist ein eigener Peer, dessen Mesh-Adresse Tofu nicht kennt. Die Verknüpfung von `homelab.example` mit der öffentlichen Doku ist dabei bewusst in Kauf genommen; die echten Adressen bleiben dort weiter ersetzt.

Wer als Admin-Gerät zählt: die Geräte der Gruppe `admin-workstations` (`terraform/netbird/subraum.tf`), heute das MacBook – dasselbe Gerät, das auch per SSH an die Subraum-Geräte darf. Bewusst nicht die Personengruppe `homelab_admins`: die füllt sich aus Pocket ID mit *allen* Geräten eines Admin-Users. Heute wären das keine (Calla hat keine eigenen Geräte), und ein Telefon, das sich unter einem Admin-User anmeldet, wäre sofort drin. Admin ist hier ein Gerät, das in Tofu mit Namen steht, nicht eine Anmeldung. Ein weiteres Gerät in `admin-workstations` bekommt damit auch SSH auf die Subraum-Geräte – dasselbe Vertrauen. Tofu liest die Mitglieder bei jedem `just netbird-apply` und schreibt Name → Adresse nach `terraform/netbird/admin-peers.json`; `flake.nix` übergibt die Adressen an `services.docsSite.allow`. Ein neues Admin-Gerät kommt also nach `just netbird-apply`, Commit und Deploy von aldrin und ursalia hinein.

Warum in nginx und nicht in einer NetBird-Policy: `to-servers` erlaubt den Geräten aller Personen (`homelab` und `homelab_admins`) alle Ports auf allen Servern. NetBird-Policies können nur erlauben, nicht verbieten – eine engere Regel für die Doku änderte nichts, solange `to-servers` so bleibt. `to-servers` auf eine Port-Liste je Dienst einzuschränken, ginge, berührt aber jeden Dienst auf jedem Server für eine statische Seite. Die Doku über einen eigenen Port abzutrennen, scheitert am selben Punkt. Die Policy entscheidet also weiter, welche Hosts ein Gerät erreicht, und nginx, wer die Doku bekommt.

Warum keine Anmeldung (Pocket ID): Die Doku liest man, wenn etwas kaputt ist. Ein Login hinge sie an zummi (Pocket ID) und, über den Reverse Proxy, an grammi – genau die Hosts, von denen sie unabhängig sein soll. Eine Adressliste braucht zur Laufzeit nichts außer nginx auf dem Host selbst.

Was das bewusst in Kauf nimmt:

- Die Liste ist so aktuell wie das letzte `just netbird-apply` mit anschließendem Deploy. Ein neues Admin-Gerät sieht bis dahin 403, ein entferntes Gerät kommt bis dahin noch hinein.
- Die Adresse steht für das Gerät, nicht für die Person: Wer ein Admin-Gerät benutzt, liest die Doku.
- Leer darf die Liste nicht werden – `just netbird-admin-peers` verweigert eine leere Ausgabe, und das Modul bricht den Build mit einer Assertion ab, statt still allen 403 zu geben.
- Nur IPv4, wie das Mesh. Mit dem IPv6-Overlay kommen die v6-Adressen in dieselbe Liste.

Deployment & Betrieb

Die Pull-Hosts (zummi, aldrin, grammi, ursalia) halten `/etc/nixos` als Checkout des privaten GitHub-Repos und bauen sich selbst. cuckoosegg wird gepusht, zorlock lokal per `darwin-rebuild` umgestellt.

Host	Manueller Deploy	Automatisch
zummi, aldrin, grammi, ursalia	`just ship <host> "<Nachricht>"` = commit, push, `git pull` + `nixos-rebuild switch` auf dem Host	nächtliches Auto-Upgrade, Reboot-Fenster 04:00–06:00
cuckoosegg	`just deploy-push cuckoosegg` (gebaut auf ursalia, kopiert zum Ziel)	ursalia pusht täglich um 06:30
zorlock	`just deploy-zorlock` (`darwin-rebuild switch` mit sudo)	keins

Nächtliches Upgrade

`modules/auto-upgrade-git.nix` aktualisiert `nixpkgs`, `nixpkgs-unstable` und `sops-nix`, baut, schaltet um und pusht die neue `flake.lock` mit dem Deploy-Key des Hosts zurück nach GitHub. Der GitHub-Host-Key ist gepinnt, weil niemand zusieht. Ein Konfigurationsfehler, der erst beim Kaltstart auffällt, legt den Host also um 04:00 lahm – bei den VMs gibt es dafür die Konsole des Anbieters.

Monitoring

Alles landet auf grammi: **vmagent** auf jedem Host schickt Metriken an VictoriaMetrics (`:8428`), **Vector** journald- und nginx-Logs an VictoriaLogs (`:9428`), beides über das Mesh. Grafana (`metrics`) und logchef (`logs`) liegen hinter dem Reverse Proxy mit SSO. Die Exporter je Host kommen aus `modules/metrics.nix`; was keinen eigenen Endpunkt hat, geht über den textfile-Collector von node_exporter (`/var/lib/prometheus-node-exporter-text-files`), geschrieben von kleinen Timern oder Diensten:

Textfile-Metrik	Host	Quelle
`tailscale_*`	Hosts mit Tailscale	`tailscale metrics write`, alle 30 s
`fwupd_updates_*`	ursalia	`fwupdmgr get-updates` nach jedem Metadaten-Refresh
`wifi_link_*`	ursalia	`iw dev wlp2s0f0 link` (PHY-Modus, Breite, MCS, NSS), alle 30 s
`node_wifi_*`, `wifi_link_*`	zorlock	`system_profiler SPAirPortDataType -json` alle 60 s, dieselben Namen wie auf ursalia (unter macOS ohne Empfangsrate, Retries und eigenen PHY-Modus für den Empfang; SSID/BSSID schwärzt macOS ohne Ortungsdienste auch für root, im Dashboard „n/a“), dazu `wifi_link_noise_dbm`
`netbird_expected_peers`	grammi	`terraform/netbird/expected-peers.json`, beim Bauen ins System übernommen

Die Dashboards liegen als JSON in `hosts/grammi/dashboards/` und werden aus dem Repo provisioniert (Änderungen nur per Commit + Deploy von grammi). `topology.json` erzeugt `just topology`.

Dashboard	Zeigt
Hosts Overview	Scrape-Ziele, CPU/RAM/Platte/Netz je Host, fehlgeschlagene Units, Zertifikate, Backups (btrbk, restic), ausstehende Firmware, WLAN-Band/-Signal
Topology	Hosts, Exporter und Datenflüsse als Graphviz-Diagramm
NetBird	Erreichbarkeit der Control Plane je Host; Peers je Host als Soll (Tofu) / known (Network Map) / connected P2P / connected Relay, dazu Abweichungen; Verlauf der Verbindungen, P2P-Latenz, Sync- und Aufbauzeiten; netbird-server (Signal, Relay, gRPC, API)
WLAN	Band von ursalia und zorlock (2,4 / 5 / 6 GHz) über die Zeit, Kanal, Signal, PHY-Modus, Linkrate, Verkehr, MCS, Retries, Verbindungsdauer

Dashboard	Zeigt
Disk Health	SMART/NVMe von ursalia: Status, kritische Warnungen, Verschleiß, Reserve, Temperatur, geschriebene Daten
Paperless	Zustand (Celery, Redis, DB, Index, Classifier), Dokumente, Inbox, fehlgeschlagene Tasks, Speicher, Log-Warnungen
paperless-gpt	OCR- und Vorschlags-Rückstau, OCR-Zeit je Seite, LLM-Anfragen, -Dauern, -Tokens, -Fehler; GPU und Speicher von zorlock
Web & Security	Anfragen an nginx und Traefik, Statuscodes, Dauern; CrowdSec-Bans, -Alerts, Parser-Treffer
Garage (S3 storage)	Garage auf aldrin: Cluster-Zustand, Platte, S3-Anfragen/-Fehler/-Dauer, Resync- und GC-Warteschlangen
VictoriaMetrics & VictoriaLogs	Ingest-Rate, Plattenbelegung, Abfragedauer, aktive Serien
zorlock (Mac mini)	mactop: Thermal State, CPU/GPU, Leistung, Temperaturen, Speicher, Lüfter; Ollama-Anfragen, -Geschwindigkeit, Ladezeiten der Modelle

Soll gegen known (NetBird): „expected“ berechnet OpenTofu aus den Policies mit der Gruppenmitgliedschaft beim letzten `just netbird-apply` (Beispiel: aldrin 6, grammi 8, cuckoosegg 4). „known“ ist, was der Management-Server tatsächlich verteilt. Ein später hinzukommendes Gerät erhöht known sofort, das Soll erst nach dem nächsten Apply + Deploy von grammi – gewollt, damit ein neuer Peer auffällt. Das Modell ignoriert Posture Checks; siehe Kopf von `expected-peers.tf`.

Grafana-Falle: State-Timelines mit `mergeValues` und Farbmodus „thresholds“ machen aus Zahlen Schwellwert-Stufen („1+“, „< 1“), bevor die Wert-Zuordnungen greifen. Farbmodus „fixed“ verwenden und Text und Farbe in die Zuordnungen legen.

Neuen Host aufnehmen

Ein Host steht an mehreren Stellen im Repo, weil NixOS und OpenTofu bewusst keine gemeinsame Host-Liste haben (siehe Was bewusst nicht verdrahtet ist). Diese Anleitung geht sie der Reihe nach durch.

1. Maschine bereitstellen

Gemietete VMs werden beim Anbieter von Hand angelegt; Bare Metal und die Subraum-VM folgen den Installationsanleitungen unter `docs/install-*.md`. Am Ende steht ein NixOS mit SSH-Host-Key – aus dem entsteht gleich der age-Schlüssel des Hosts.

2. Konfiguration anlegen

- `hosts/<host>/configuration.nix` und `hardware-configuration.nix` anlegen.
- In `flake.nix` unter `nixosConfigurations` eintragen, mit expliziter Modulliste. Was bewusst fehlt, bekommt einen Kommentar mit Begründung daneben.

3. Überall eintragen, wo Hosts aufgezählt sind

Datei	Eintrag
`flake.nix`	`nixosConfigurations.<host>`
`inventory.json`	`hosts.<host>` mit `"managed": true` – Art, Site, Owner, Entsperren (siehe Inventar)
`justfile`	`hosts := "…"` (nur Pull-Hosts, für `ship-all`)
`terraform/netbird/main.tf`	`local.hosts`: Host → Setup-Key-Gruppe (`servers` o. ä.)

Datei	Eintrag
`terraform/desec-tokens/main.tf`	`local.hosts`: erlaubte Quelladressen, `acme_names`
`terraform/github/main.tf`	`local.hosts`: Deploy-Key nur lesend oder mit Schreibrecht

4. sops-Empfänger

Den age-Schlüssel aus dem SSH-Host-Key des neuen Hosts ableiten (`ssh-to-age` auf `/etc/ssh/ssh_host_ed25519_key.pub`) und in `.sops.yaml` eintragen: einen Anker unter `keys` und eine `creation_rule` für `hosts/<host>/(secrets|netbird|desec|github)\.yaml$` mit dem Host und `admin` als Empfängern. Der öffentliche Host-Key selbst kommt in `inventory.json` (`sshHostKeys`, oder später per `just inventory-collect`); `checks.inventory-sops` gleicht beides ab.

5. Schlüssel und Tokens erzeugen

Rezept	Ergebnis	Hinweis
`just netbird-key <host>`	`hosts/<host>/netbird.yaml`	Einmal-Setup-Key, **24 h gültig** – direkt danach deployen
`just desec-key <host>`	`hosts/<host>/desec.yaml`	nur für Hosts mit eigenen Zertifikaten
`just github-key <host>`	`hosts/<host>/github.yaml` + `github-deploy.pub`	nur für Pull-Hosts

Rezepte, die auf dem MacBook sops brauchen, laufen mit der 1Password-Anmeldung davor (siehe just-Rezepte).

6. Deployen und prüfen

- Erster Deploy innerhalb von 24 h nach `netbird-key`. Der Host registriert sich beim Start selbst im Mesh; Tofu findet ihn über `data "netbird_peer"` am Namen.
- `just topology` ausführen und das Ergebnis committen – sonst schlägt `nix flake check` fehl.
- `just inventory-collect` (Host-Key, Interfaces), `network/ipv4` von Hand ergänzen, dann `just inventory-test`; auf dem MacBook `just known-hosts`.
- App-Secrets erst eintragen, wenn der Host läuft: Secrets eintragen.
- Doku nachziehen: Hosts & Rollen, Services.

Neuen Dienst veröffentlichen

Dienste heißen nach ihrer Funktion (`rezepte`, nicht `mealie`); neue Webdienste auf grammi nutzen PostgreSQL. Zuerst entscheiden, wie der Dienst erreichbar sein soll – davon hängt ab, welche Dateien sich ändern.

Erreichbarkeit	Für	Beispiele
Nur im Mesh	Admin-Oberflächen, Dienste für die eigenen Geräte	`s3-admin`, `llm`, `dokumente`
Öffentlich hinter dem Reverse Proxy mit SSO	Web-Apps, die auch von unterwegs ohne NetBird-App erreichbar sein sollen	`rezepte`, `grist`, `metrics`
Direkt öffentlich	Dienste mit eigenem Login für Maschinen oder alle	`auth`, `s3.aldrin`

Nur im Mesh

1. In `terraform/netbird/mesh-services.tf` eine Zeile unter `local.mesh_services`: Name → Host(s). Daraus entsteht eine eigene NetBird-DNS-Zone mit A-Record(s) auf die Mesh-Adresse(n); öffentliches DNS kennt den Namen nicht.
2. `just netbird-apply` – neue Zonen kommen seit NetBird 0.81 ohne Neustart bei den Clients an.
3. Auf dem Host einen nginx-vHost mit `enableACME`, `acmeRoot = null` (DNS-01 über deSEC) und `allow 192.0.2.0/24; deny all;`.
4. Steht der Dienst auf ursalia oder zorlock: den Namen in `acme_names` des Hosts in `terraform/desec-tokens/main.tf` ergänzen und `just desec-apply` – deren Tokens dürfen nur genau diese TXT-Records schreiben.
5. Host deployen, `just topology` (neuer vHost).

Öffentlich hinter dem Reverse Proxy

Für Dienste auf grammi:

1. Eintrag in `hosts/grammi/proxy-services.json`: Name → App, Port, Pocket-ID-Gruppen. Tofu (`reverse-proxy.tf`) und `topology.nix` lesen dieselbe Datei.
2. Der Dienst lauscht auf allen Adressen; öffentlich sind nur 80/443 offen, `nb-mesh` ist vertrauenswürdig.
3. In DNSControl einen CNAME `<name>` → `proxy.homelab.example` pushen – einige Minuten vor dem Apply, sonst cacht der Resolver NXDOMAIN.
4. Erst grammi deployen, dann `just netbird-apply`. Das ruft am Ende `just netbird-proxy-certs` auf: Es fragt jeden neuen Namen einmal von außen ab, damit der Proxy sein Zertifikat holt, und startet `netbird-proxy` danach einmal neu – sonst hängt das Dashboard bei „Issuing certificate…“ (NetBird-Diskussion #7118).

Dienste auf anderen Hosts bekommen eine eigene Ressource in `terraform/netbird/reverse-proxy.tf` mit dem Host als Peer-Ziel (Beispiele: `dokumente` auf ursalia, `doku` auf aldrin). Auch hier: CNAME vorher pushen, `just netbird-apply` erledigt die Zertifikate.

Direkt öffentlich

nginx-vHost auf dem Host (ACME per DNS-01), A/AAAA-Records in DNSControl, CrowdSec liest die nginx-Logs mit. Nur für Dienste, die ohnehin eine eigene Anmeldung haben.

Danach

- Doku: Services, bei neuem Host-Verzeichnis-Inhalt auch Inhalt von hosts/<host>/.
- Dual-Stack, wo es geht (A und AAAA).

Secrets eintragen

App-Passwörter, OIDC-Clients und Ähnliches liegen in `hosts/<host>/secrets.yaml`, verschlüsselt für den Host und den Recovery-Key `admin`. Das MacBook kann diese Dateien nicht entschlüsseln – deshalb wird auf dem Host editiert (Hintergrund: Secrets, Netzwerk & Zugriff).

Ablauf

1. `just sync <host>` – sonst liegt auf dem Host ein veralteter Stand von `secrets.yaml`, und `pull-secrets` holt ihn zurück ins Repo.
2. `just secrets <host>` – öffnet sops auf dem Host (mit dessen Host-Key, ohne 1Password-Anmeldung) und holt das Ergebnis danach per `pull-secrets` ins Repo.
3. Werte immer in Anführungszeichen: `key: "wert"`, einfache Anführungszeichen, wenn der Wert `"` oder `\` enthält. Ein unquotierter Wert kann von YAML als Zahl oder Bool gelesen werden; sops-nix bricht dann `setupSecrets` ab und **kein** Secret dieser Generation wird installiert.
4. Die NixOS-Seite (`sops.secrets."<name>"`) im Host-Modul ergänzen, committen, deployen: `just ship <host> "<nachricht>"`.

Was nicht von Hand editiert wird

Datei	Erzeugt von
`hosts/<host>/netbird.yaml`	`just netbird-key <host>`
`hosts/<host>/desec.yaml`	`just desec-key <host>`
`hosts/<host>/github.yaml`	`just github-key <host>`
`terraform/*/secrets.yaml`	sops auf dem MacBook mit dem Workstation-Key aus 1Password

Alles, was auf dem MacBook sops braucht, läuft mit vorangestellter Anmeldung – siehe just-Rezepte.

Schlüssel rotieren

Was sich wie erneuern lässt und was danach zu tun ist. Die Rezepte ersetzen den alten Schlüssel jeweils sofort – deshalb direkt danach committen und deployen.

Schlüssel	Rezept	Danach	Wann
NetBird-Setup-Key eines Hosts	`just netbird-key <host>`	Deploy innerhalb von 24 h	bei Neuinstallation oder verlorenem Client-State
deSEC-Token eines Hosts	`just desec-key <host>`	sofort deployen, dann `just desec-verify`	bei Verdacht, sonst nie
GitHub-Deploy-Key eines Hosts	`just github-key <host>`	commit + deploy	bei Verdacht
Tofu-Token für NetBird (Service-User `terraform`)	`just netbird-renew-token`	–	jährlich
App-Secrets	`just secrets <host>` (siehe Secrets eintragen)	deploy	bei Verdacht

SSH-Host-Key (= sops-Empfänger)

Der age-Schlüssel eines Hosts ist aus seinem SSH-Host-Key abgeleitet. Ein neuer Host-Key heißt deshalb auch:

1. Neuen age-Schlüssel per `ssh-to-age` ableiten und in `.sops.yaml` ersetzen.
2. Die Dateien des Hosts mit `sops updatekeys` neu verschlüsseln – dafür braucht es einen Schlüssel, der sie noch lesen kann (der alte Host-Key oder `admin`).
3. Den neuen öffentlichen Key in `inventory.json` übernehmen: `just inventory-collect --force <host>`. `nix flake check` (`checks.inventory-sops`) schlägt fehl, bis Inventar und `.sops.yaml` übereinstimmen.
4. Committen und deployen, bevor der alte Key verschwindet – die known_hosts aller Hosts kommt aus dem Inventar. Auf dem MacBook: `just known-hosts`.

Nach einer Rotation

- Alte Schlüssel beim Gegenüber löschen (GitHub, deSEC und NetBird erledigt das Rezept per Tofu).
- `just docs`, falls sich Namen oder Abläufe geändert haben.

Repository-Struktur

Ein Flake, `nixosConfigurations` für fünf Hosts plus `darwinConfigurations.zorlock`. Jeder Host bekommt eine explizite Modulliste in `flake.nix`; was fehlt, fehlt mit Begründung im Kommentar daneben.

Pfad	Inhalt
`flake.nix` / `flake.lock`	Inputs, Host-Definitionen, Pakete, `lib.topology`, `lib.inventory`, `checks`
`modules/`	17 geteilte Module: base, nix, ssh, firewall, networking (ifstate), acme, crowdsec, tailscale, netbird, metrics/metrics-shipping, logging/log-shipping, qemu-guest, deploy, auto-upgrade-git, docs-site
`hosts/<host>/`	`configuration.nix`, Hardware, Dienst-Module, sops-Dateien (`secrets.yaml`, `netbird.yaml`, `desec.yaml`)
`pkgs/`	Eigene Pakete: garage-ui, logchef, paperless-gpt, prometheus-paperless-exporter, ollama-metrics-proxy
`terraform/netbird/`	OpenTofu: NetBird-Account, Policies, DNS-Zonen, Reverse-Proxy-Dienste, Personen/Gruppen (inkl. Pocket ID)
`terraform/desec-tokens/`	OpenTofu: eng gescopte deSEC-Tokens pro Host für DNS-01
`terraform/github/`	OpenTofu: die Repos `nixlab` (privat) und `nixlab-docs` (öffentlich), Deploy-Keys pro Host
`topology.nix`	Generiert das Topologie-Dashboard für Grafana aus den Host-Configs
`inventory.json` / `inventory.nix`	Lab-Inventar (jede Maschine: Art, Site, Uplink, Beziehungen, Entsperren) und seine Prüfung, siehe Inventar
`inventory.schema.json`	JSON-Schema zum Bearbeiten von `inventory.json`, generiert
`tests/`	Tests, die `nix flake check` auswertet (`inventory.nix`)
`justfile`	Alle Bedienabläufe: deploy, secrets, netbird-*, desec-*
`docs/`	Diese Projektdokumentation: Seiten unter `site/{de,en}/`, Zensical-Konfiguration, Build (`build.sh`, `site-src.sh`, `assemble.py`, `gen-reference.py`), Paket der privaten Site (`package.nix`) und was sie sonst von Dritten laden würde – Schriften, glightbox (`assets.nix`), Installationsanleitungen (ursalia, cuckoosegg), Upstream-Issue-Entwürfe

Pfad	Inhalt
`dnsconfig.js`	Symlink ins separate DNSControl-Repo – öffentliche DNS-Records leben dort
`.sops.yaml`	age-Empfänger pro Host, `admin` (Offline-Recovery) und `workstation` (1Password)

Wo ändere ich was?

Welche Datei für welche Änderung zuständig ist, wie oft das vorkommt und was danach zu tun ist.

Was	Wo	Wie oft	Danach
Dienst, Paket oder Option auf einem Host	`hosts/<host>/*.nix`	häufig	`just ship <host> "…"`
Etwas für alle Hosts	`modules/*.nix`	gelegentlich	`just ship-all "…"`; neues Modul: öffentlich oder in `docs/modules-private.txt`?
Name hinter dem Reverse Proxy (grammi)	`hosts/grammi/proxy-services.json`	selten	CNAME in DNSControl, grammi deployen, `just netbird-apply`
Name nur im Mesh	`terraform/netbird/mesh-services.tf`	selten	`just netbird-apply`
Wer die private Doku lesen darf	Gruppe `admin-workstations` (`terraform/netbird/subraum.tf`, Peer-Namen)	selten	`just netbird-apply`, `admin-peers.json` committen, aldrin und ursalia deployen
NetBird-Policies, Gruppen, Personen, Routen	`terraform/netbird/*.tf`	selten	`just netbird-plan`, `just netbird-apply`
Öffentliche DNS-Records	DNSControl-Repo (`dnsconfig.js`)	selten	dort pushen
Was ein deSEC-Token darf	`terraform/desec-tokens/main.tf`	selten	`just desec-apply`
Repos, Deploy-Keys	`terraform/github/`	selten	`just github-plan`, `just github-apply`
Version eines automatisch aktualisierten Inputs	`flake.lock`	nachts automatisch	–
Version eines separat gepinnten Inputs	`flake.lock` (`nix flake update <input>`)	von Hand	Hosts deployen, die ihn nutzen
Grafana-Dashboard	`hosts/grammi/dashboards/*.json`	gelegentlich	grammi deployen
Topologie (neuer Host, Exporter, vHost)	`topology.nix` erzeugt sie	bei Bedarf	`just topology`, committen
Maschinendaten: Art, Hardware, Site, Uplink, Strom, Entsperren, Netze, Interfaces, Host-Keys	`inventory.json`	mit jedem neuen Gerät	`just inventory-collect`, `just inventory-test` (Inventar)
Inventar-Format (Felder, Gerätearten)	`spec` in `inventory.nix`	selten	`just inventory-schema`, Testfall, Doku
Secrets eines Hosts	`hosts/<host>/secrets.yaml`	selten	Secrets eintragen
Diese Dokumentation	`docs/site/{de,en}/`	mit jeder Änderung	`just docs`

Inhalt von hosts/<host>/

Was in einem Host-Verzeichnis liegt und wer es schreibt. Nicht jeder Host hat jede Datei.

Datei	Inhalt	Geschrieben von
`configuration.nix`	Host-Konfiguration: Netz, Dienste, vHosts	von Hand
`hardware-configuration.nix`	Hardware, Dateisysteme	`nixos-generate-config`, danach von Hand
`disk-config.nix`	Partitionierung (disko, nur ursalia)	von Hand
weitere `*.nix`	ein Dienst pro Datei, z. B. `backup.nix`, `samba.nix`, `netbird.nix`, `traefik.nix`	von Hand
`secrets.yaml`	App-Secrets, verschlüsselt für Host + admin	`just secrets <host>`
`netbird.yaml`	NetBird-Setup-Key	`just netbird-key <host>` (Tofu)
`desec.yaml`	deSEC-Token für DNS-01	`just desec-key <host>` (Tofu)
`github.yaml`	privater Teil des Deploy-Keys	`just github-key <host>`
`github-deploy.pub`	öffentlicher Teil, von `terraform/github` gelesen	`just github-key <host>`
weitere `*.yaml`	sops-Dateien einzelner Dienste, z. B. `netbird-server.yaml`, `netbird-proxy.yaml` (grammi), `push-cuckoosegg.yaml` (ursalia)	ihre Rezepte, z. B. `just netbird-server-secrets`, `just netbird-proxy-token`
`push-key.pub`	öffentlicher Schlüssel für den Push-Deploy (nur cuckoosegg)	`just cuckoosegg-push-key`
`dashboards/*.json`	Grafana-Dashboards (nur grammi)	von Hand, `topology.json` per `just topology`
`proxy-services.json`	Dienste hinter dem Reverse Proxy (nur grammi)	von Hand, gelesen von Tofu und `topology.nix`

Inputs und ihre Pins

Nur `nixpkgs`, `nixpkgs-unstable` und `sops-nix` laufen im nächtlichen Auto-Upgrade mit (`updateInputs` in `modules/deploy.nix`). Alles, was bei einem kaputten Release das ganze Lab aussperren würde, ist separat gepinnt und wird von Hand gebumpt.

Input	Branch / Tag	Wer nutzt es	Nachts mit?
`nixpkgs`	nixos-26.05	alle NixOS-Hosts	ja
`nixpkgs-unstable`	nixos-unstable	grammi (Grist), ursalia (Paperless 3.x), zorlock (mactop)	ja
`sops-nix`	main	alle	ja
`nixpkgs-netbird`	nixos-unstable	NetBird-Server und alle Clients	nein – Server zuerst, dann Clients
`nixpkgs-pocketid`	nixos-26.05	zummi (pocket-id-Paket)	nein
`disko`	main	ursalia (alle Mounts)	nein
`lanzaboote`	v1.1.0	ursalia (Secure Boot)	nein
`nixpkgs-darwin`, `nix-darwin`	26.05-darwin	zorlock	nein (kein Auto-Upgrade)

`nix flake check` prüft zusätzlich, dass `hosts/grammi/dashboards/topology.json` zu dem passt, was `topology.nix` gerade erzeugt – ein neuer Exporter ohne `just topology` lässt den Check fehlschlagen.

Hosts & Rollen

Drei gemietete VMs tragen die öffentlichen Dienste, ursalia und zorlock stehen zu Hause, cuckoosegg im Subraum. Alle sind über `<host>.peer.homelab.example` im NetBird-Mesh erreichbar; root-SSH gibt es nur dort. Art, Site, Uplink und Entsperren jeder Maschine – auch der Router, des KVM und der Appliances – stehen im Inventar.

Host	Plattform	Rolle	Besonderheiten
grammi	NixOS, gemietete VM	NetBird-Server (`mesh.homelab.example`) + Reverse Proxy, Traefik, Monitoring (VictoriaMetrics/-Logs, Grafana, logchef), Mealie, Grist, PostgreSQL	Zentrale Senke für Metriken und Logs aller Hosts
zummi	NixOS, gemietete VM	Pocket ID (`auth.homelab.example`) – Login für alle Dienste und NetBird	pocket-id separat gepinnt
aldrin	NixOS, gemietete VM	Garage S3 (`s3.aldrin.homelab.example`), garage-ui (`s3-admin`), Backup-Ziel für restic, Doku-Site (`doku`)	Fork-Paket garage-ui mit Subpath-Support
ursalia	NixOS, Bare Metal (ThinkCentre M720q)	Paperless-ngx 3 + paperless-gpt (`dokumente`), Doku-Site (`doku`), Samba für den Scanner, Routing-Peer ins Heim-LAN, Build-Host für cuckoosegg	LUKS + btrfs via disko, TPM-Unlock, Secure Boot (lanzaboote), WLAN (Intel BE200), kein CrowdSec
cuckoosegg	NixOS, VM auf dem Subraum-Proxmox	NetBird-Routing-Peer in die Subraum-Netze	Gruppe `subraum-devices` statt `servers`, kein Repo/Deploy-Key, wird per Push deployed
zorlock	macOS + nix-darwin, Mac mini M4	Ollama auf der Apple-GPU (`llm`), genutzt von paperless-gpt	Keins der `modules/*.nix`; FileVault-Unlock nach Reboot per LAN-SSH

Inventar

`inventory.json` führt jede Maschine des Labs, ob dieser Flake sie konfiguriert oder nicht: was für ein Gerät es ist (Hersteller, Modell, Seriennummer), wo es steht, wie dieser Ort ans Internet kommt, worauf es läuft oder worin es steckt, woher es Strom bekommt, wie seine Platte nach einem Reboot entsperrt wird, seine Netze, Interfaces und Adressen und seine SSH-Host-Keys. Das sind die Angaben, die in keiner NixOS-Config stehen. Genutzt werden heute die Host-Keys: Die known_hosts jedes Hosts kommt von hier, und `nix flake check` gleicht sie mit den age-Schlüsseln in `.sops.yaml` ab. Der Rest soll später der Gruppierung in der Topologie und den Abhängigkeiten im Alerting dienen (der Uplink ist weg, also nicht für jeden Host dahinter alarmieren).

Datei	Aufgabe
`inventory.json`	Die Daten. JSON, damit Nix (`builtins.fromJSON`) und OpenTofu (`jsondecode`) sie beide ohne Umweg lesen.
`inventory.nix`	Format (`spec`), Validierung, abgeleitete Sicht, Generator für das JSON-Schema. In `flake.nix` als `lib.inventory`.
`inventory.schema.json`	JSON-Schema für den Editor (Vervollständigung, Hover-Hilfe, Fehler direkt im Text). Generiert, nie von Hand ändern.

Datei	Aufgabe
`tests/inventory.nix`	Tests für die Validierung: ein vollständiges Beispielinventar und ein Fall je Regel.
`flake.nix`	`lib.inventory`, die Checks und `programs.ssh.knownHosts` für jeden Host (NixOS und nix-darwin).

Faustregel für den Inhalt: **nur, was sich nicht aus einer Config ableiten lässt.** OS, Architektur, CPU/RAM, Mesh-Adresse, Exporter und Backup-Ziele bleiben draußen; die liest `topology.nix` aus `nixosConfigurations`. Jede Beziehung steht **nur in einer Richtung** drin; die Gegenrichtung wird abgeleitet, damit beide nicht auseinanderlaufen können.

Aufbau

Fünf Abschnitte, jeder ein Objekt aus Einträgen mit ID als Schlüssel. IDs bestehen aus Kleinbuchstaben, Ziffern und Bindestrichen (unverändert als Graphviz-Knoten oder Label verwendbar).

```
owners   ◀──────── hosts.owner
uplinks  ◀──────── sites.uplinks[]
sites    ◀──────── hosts.site, networks.site
networks ◀──────── hosts.interfaces.<name>.network
hosts    ◀──────── hosts.runsOn / inside / power / manages[] / unlock.tang[], networks.dhcpServer
```

Ein Schlüssel `"$schema": "./inventory.schema.json"` ganz oben ist erlaubt und wird ignoriert; er sagt dem Editor, wo das Schema liegt.

owners

Wer ein Gerät administriert: entscheidet, wen man anruft und was man ändern darf.

Feld	Typ		Bedeutung
`name`	String	Pflicht	Anzeigename
`contact`	String		Ansprechpartner, wenn Geräte dieses Owners betroffen sind
`note`	String		Freitext

uplinks

Ein Internetanschluss. Mehrere Sites können sich einen teilen (selten), eine Site kann mehrere haben (primär und Backup).

Feld	Typ		Bedeutung
`isp`	String		Provider; im Rechenzentrum der Hoster
`asn`	Ganzzahl ≥ 1		AS-Nummer des Provider-Netzes, ohne „AS“
`medium`	`fiber` `dsl` `cable` `lte` `satellite` `datacenter`		Anschlussart
`bandwidth`	`{ "down": int, "up": int }`		Gebuchte Bandbreite in Mbit/s, beide Werte Pflicht
`ipv4`	`public` `cgnat` `none`		Was der Anschluss für IPv4 hergibt
`ipv4Static`	Boolean		Feste öffentliche IPv4-Adresse (nur bei `public` sinnvoll)
`ipv6`	Boolean		Natives IPv6
`ipv6PrefixLength`	Ganzzahl 1–128		Größe des delegierten IPv6-Präfixes (56 für ein /56); nur mit `ipv6`

Feld	Typ		Bedeutung
`ipv6Static`	Boolean		Festes IPv6-Präfix; nur mit `ipv6`
`note`	String		Freitext

Alle Felder sind optional, damit ein Anschluss schon eingetragen werden kann, bevor die Details bekannt sind.

sites

Ein Ort, an dem Geräte stehen.

Feld	Typ		Bedeutung
`city`	String	Pflicht	Stadt
`kind`	`residential` `hackerspace` `datacenter`	Pflicht	Art des Orts
`provider`	String		Wer den Ort betreibt (Hoster, Verein); bei einer Wohnung leer
`uplinks`	Liste von Uplink-IDs	Pflicht, ≥ 1	Internetanschlüsse, **der primäre zuerst**, Backups danach
`note`	String		Freitext

hosts

Jedes Gerät mit einer Adresse, auf die es ankommt: Server, VMs, Router, APs, KVMs, PDUs, Appliances.

Feld	Typ		Bedeutung
`kind`	siehe Gerätearten	Pflicht	Was das Gerät ist
`vendor`	String		Hersteller (Lenovo, Apple, GL.iNet). Nicht bei `vm`/`container`
`model`	String		Modell ohne Hersteller (ThinkCentre M720q), bei einer VM das Produkt bzw. der Tarif des Anbieters
`productId`	String		Kennung des Herstellers für Treiber und Firmware: Lenovo-Maschinentyp (`10T7…`), Apple-Modellkennung (`Mac16,10`). Nicht bei `vm`/`container`
`serial`	String		Seriennummer, im ganzen Lab eindeutig. Nicht bei `vm`/`container`
`site`	Site-ID	Pflicht*	Wo es steht. *Optional mit `runsOn` oder `inside`: dann geerbt
`owner`	Owner-ID	Pflicht	Wer es administriert
`managed`	Boolean	Pflicht	Von diesem Flake konfiguriert – muss zu `flake.nix` passen
`criticality`	`core` `service` `lab`	Standard `service`	`core`: ohne das steht das Lab; `service`: ein Dienst ist weg; `lab`: nichts hängt daran
`availability`	`always-on` `intermittent`	Standard `always-on`	`intermittent`: offline ist normal (Laptop), kein Vorfall
`unlock`	Objekt, siehe Entsperren		Weglassen bei unverschlüsselter Platte
`runsOn`	Host-ID		Nur `vm`/`container`: die Maschine, auf der es läuft
`inside`	Host-ID		Physisch in ein anderes Gerät eingebaut

Feld	Typ		Bedeutung
`power`	Host-ID		Stromquelle: PDU/USV, PoE-Switch oder der Host, in dem eine Karte steckt
`manages`	Liste von Host-IDs		Nur `kvm`: Maschinen, auf die es Out-of-Band-Zugriff gibt
`sshHostKeys`	Liste von Strings		Öffentliche SSH-Host-Keys, `"<typ> <base64>"` ohne Kommentar, siehe SSH-Host-Keys
`interfaces`	Objekt: Name → Interface		Netzwerk-Interfaces, siehe Interfaces
`note`	String		Freitext

Gerätearten { #kinds }

Art	Für
`server`	Physische Maschine mit Diensten
`hypervisor`	Physische Maschine, die VMs/Container für andere betreibt
`vm`, `container`	Virtuell; die einzigen Arten, die `runsOn` haben dürfen
`router`, `switch`, `ap`	Netz (`ap` = WLAN-Access-Point)
`pdu`, `ups`	Strom
`kvm`	IP-KVM, extern oder als Karte im Host; die einzige Art, die `manages` haben darf
`nas`, `workstation`, `laptop`	Wie benannt
`appliance`	Alles andere mit Adresse: Apple TV, Drucker, …

Ob ein Host virtuell ist, folgt aus `kind` (`vm`, `container`) – nicht aus `runsOn`. Eine gemietete VM ist eine `vm` ohne `runsOn`: Der Hypervisor des Anbieters steht nicht im Inventar.

networks

Ein Layer-2-Netz mit seinem IPv4-Präfix: das Heimnetz, ein VLAN im Hackerspace.

Feld	Typ		Bedeutung
`name`	String	Pflicht	Bezeichnung (die ID ist der technische Schlüssel)
`site`	Site-ID	Pflicht	Wo das Netz ist
`vlan`	Ganzzahl 1–4094		802.1Q-VLAN-ID; weglassen bei einem ungetaggten Netz. Eindeutig je Site – eine andere Site darf dieselbe ID nutzen
`ipv4`	Präfix, z. B. `192.0.2.0/24`		Ohne Host-Bits
`ipv6`	Präfix, z. B. `2001:db8:1::/64`		Nur wenn es fest ist (ULA, statische GUA) – kein wechselndes Provider-Präfix
`dhcpServer`	Host-ID		Verteilt IPv4 per DHCP; muss an der Site sein und ein Interface in diesem Netz haben
`dhcpPool`	`{ "start": ip, "end": ip }`		Dynamischer Bereich; innerhalb von `ipv4`, Start ≤ Ende. Braucht `ipv4` und `dhcpServer`
`note`	String		Freitext

Ein Netz ohne Interface ist erlaubt – es kann vor seinen Geräten eingetragen werden.

Interfaces { #interfaces }

`hosts.<id>.interfaces` ordnet Interface-Namen ihre Daten zu. Der Name ist der des Betriebssystems (`eno1`, `wlp2s0f0`, `en0`); bei einer Appliance ein beliebiges Label (`lan`, `wan`). Ein VLAN-Interface auf einem Trunk (`eno1.20`) ist ein eigener Eintrag; seine VLAN-ID kommt aus seinem Netz.

```
"interfaces": {
  "eno1":    { "mac": "52:54:00:12:34:56", "medium": "ethernet", "linkLocal": "eui64",
               "network": "lan", "ipv4": { "mode": "reserved", "address": "192.0.2.10" } },
  "eno1.20": { "network": "iot", "linkLocal": "fe80::1234", "ipv4": { "mode": "dhcp" } }
}
```

Feld	Typ		Bedeutung
`mac`	`aa:bb:cc:dd:ee:ff`, klein		Hardware-Adresse. Im ganzen Lab eindeutig, keine Multicast-Adresse
`medium`	`ethernet` `wifi`		Physisches Medium
`linkLocal`	`eui64` `stable-privacy` `random` oder eine `fe80::`-Adresse		Wie die IPv6-Link-Local-Adresse entsteht
`network`	Netz-ID		Netz, in dem es hängt; an derselben Site wie der Host
`ipv4`	`{ "mode": …, "address": … }`		IPv4-Adressierung, braucht `network`
`note`	String		Freitext

Link-Local. Nur bei EUI-64 folgt die Adresse aus der MAC (`fe80::` + MAC mit umgedrehtem U/L-Bit und `ff:fe` in der Mitte); das abgeleitete `ipv6LinkLocal` wird dann berechnet. Viele Systeme nutzen stattdessen `stable-privacy` (RFC 7217: stabil, aber nicht berechenbar), auch macOS und tvOS leiten sie nicht aus der MAC ab. Ist die Adresse stabil und bekannt, trägt man sie direkt ein (`"fe80::…"`); genau das macht `just inventory-collect`, wenn sie nicht zu EUI-64 passt. `eui64` braucht eine `mac`; mit einer *lokal verwalteten* WLAN-MAC (private WLAN-Adresse) ist es ein Fehler, weil diese MAC – und damit die Adresse – meist zufällig ist. Eine lokal verwaltete Ethernet-MAC ist in Ordnung (QEMUs `52:54:00:…` ist eine).

IPv4 – `ipv4.mode`:

Modus	Bedeutung	Regeln
`reserved`	Feste DHCP-Lease per MAC (Reservierung auf dem `dhcpServer`)	`address` und `mac` Pflicht; das Netz braucht einen `dhcpServer`
`static`	Auf dem Gerät selbst eingetragen	`address` Pflicht; außerhalb des `dhcpPool`
`dhcp`	Dynamisch	keine `address`

Jede `address` liegt im `ipv4`-Präfix des Netzes, ist weder Netz- noch Broadcast-Adresse (außer bei /31, /32) und kommt je Netz nur einmal vor. Ein Netz ohne `ipv4` kann keine Adressen aufnehmen.

IPv6 steht hier nur als Link-Local und als festes Präfix je Netz: DHCPv6-Reservierungen hängen an der DUID, nicht an der MAC, und ein Provider-Präfix (SLAAC) wechselt.

SSH-Host-Keys { #ssh-host-keys }

`sshHostKeys` enthält die öffentlichen Host-Keys eines Geräts – verwaltet oder nicht – als `"<typ> <base64>"` ohne Kommentar: `ssh-ed25519`, `ecdsa-sha2-nistp256/384/521` oder `ssh-rsa`. Ein ed25519-Key genügt; andere nur, wenn das Gerät kein ed25519 hat (Dropbear, KVMs). Geprüft wird das Format, ob die Key-Daten zum Typ passen (das Base64 beginnt mit dem kodierten Typnamen) und dass kein Key doppelt vorkommt – so fiele eine geklonte VM auf, die ihren Host-Key behalten hat.

Wohin sie gehen:

Abnehmer	Wie
Jeder NixOS-Host und zorlock	`programs.ssh.knownHosts` aus `lib.inventory.knownHosts` (in `flake.nix` das Modul `inventoryKnownHosts` in jeder Host-Liste). Deploys und ursalias unbeaufsichtigter Push nach cuckoosegg brauchen kein Trust-on-first-use
Das MacBook	`just known-hosts` schreibt `~/.ssh/known_hosts.nixlab`; ssh liest sie mit `UserKnownHostsFile ~/.ssh/known_hosts ~/.ssh/known_hosts.nixlab` unter `Host *` in `~/.ssh/config`
`.sops.yaml`	`checks.inventory-sops`: Für jeden verwalteten Host muss `ssh-to-age` seines ed25519-Keys den age-Empfänger `&<host>` in `.sops.yaml` ergeben. Fängt einen Host-Key, der sich ohne `sops updatekeys` geändert hat (Neuinstallation, Rotation), und einen vertippten Key. Hosts ohne Key werden übersprungen

Namen je Key: die ID des Hosts, bei verwalteten Hosts `<host>.peer.homelab.example` (der Mesh-Name) und jede feste IPv4-Adresse seiner Interfaces (`static`/`reserved`). Ein Eintrag je Key, benannt `<host>` bzw. `<host>-<n>` bei mehreren Keys.

Von den Geräten einsammeln

`just inventory-collect` holt, was mühsam abzutippen ist:

- **Verwaltete Hosts**, über das Mesh (root, zorlock als `admin`):
  - Host-Keys per `ssh-keyscan` (nur der ed25519-Key, wenn es einen gibt);
  - je Interface Name, MAC, `ethernet`/`wifi` und Link-Local – `eui64`, wenn die Adresse zur MAC passt, sonst die Adresse selbst. Unter Linux jedes Interface mit einem Gerät dahinter (Hardware oder virtio; kein `lo`, Mesh, Tunnel, Bridges, veths), unter macOS die Hardware-Ports ohne Thunderbolt;
  - bei Hardware, die nicht virtuell ist: `vendor`, `model`, `productId`, `serial`. Unter Linux aus `/sys/class/dmi/id` (Lenovo: Modell aus `product_version`, Maschinentyp aus `product_name`; alle anderen: Modell aus `product_name`), unter macOS aus `system_profiler` (Modell als „Mac mini (M4, 16 GB)“, die Modellkennung als `productId`). Platzhalter wie „To be filled by O.E.M.“ fallen weg.
- **Nicht verwaltete Geräte**: nur Host-Keys, an ihrer ersten festen IPv4-Adresse – `network`/`ipv4` also vorher eintragen.

Aufruf	Wirkung
`just inventory-collect`	Alle Geräte; **ergänzt** nur, was fehlt, ein abweichender Wert wird ausgegeben und bleibt
`just inventory-collect ursalia beryl`	Nur diese Geräte
`just inventory-collect --force ursalia`	Übernimmt bei Abweichungen den Wert des Geräts und gibt jeden überschriebenen Wert aus

Ein geänderter Host-Key soll eine Entscheidung sein, daher `--force` – am besten mit dem Host dazu. Auch `--force` rührt nie an, was das Gerät nicht meldet: `network`, `ipv4`, Notizen und Interfaces, die es nicht mehr hat (die von Hand entfernen).

Beziehungen

Feld	Bedeutung	Darf zeigen auf	Abgeleitete Gegenrichtung
`runsOn`	Virtualisierung	`vm` → `hypervisor`; `container` → `hypervisor`, `server`, `nas`, `vm`	`guests`
`inside`	Physischer Einbau (PCIe-Karte, Modul)	jeden Host	`contains`
`power`	Stromversorgung	jeden Host	`powers`
`manages`	Out-of-Band-Zugriff (am KVM eingetragen)	jeden Host außer sich selbst	`managedBy`
`unlock.tang`	Tang-Server für NBDE	jeden Host außer sich selbst	`tangClients`
`interfaces.*.network`	Zugehörigkeit zu einem Netz	ein Netz an der Site des Hosts	`networks.<id>.interfaces`
`networks.dhcpServer`	DHCP für ein Netz	einen Host mit Interface darin	–

`runsOn`, `inside` und `power` bilden zusammen den Abhängigkeitsgraphen: Ohne was kann ein Host nicht arbeiten? Er muss zyklenfrei sein; das abgeleitete `upstream` eines Hosts ist seine transitive Hülle. `runsOn` und `inside` schließen sich aus.

`inside` sagt nichts über Strom: Ob eine PCIe-KVM-Karte vom Slot ihres Hosts lebt (Standby-Strom) oder von PoE/USB, sagt `power`. Eine Karte, die nur aus dem Slot zieht, bekommt `"power"` auf ihren Host. Dann deutet „Host und KVM beide weg“ auf Strom oder Netz, „Host weg, KVM noch da“ aufs OS – und der KVM ist der Weg hinein.

Entsperren

```
"unlock": { "methods": ["tpm", "nbde"], "threshold": 1, "tang": ["nas"], "fallback": "remote-
        ssh" }
```

Feld	Typ		Bedeutung
`methods`	Liste aus `tpm`, `nbde`	Pflicht (darf leer sein)	Automatische Methoden. Leer: nur ein Mensch kann entsperren
`threshold`	Ganzzahl ≥ 1	Standard 1	Wie viele Methoden gelingen müssen – der `sss`-Schwellwert von Clevis. 1 = eine reicht, 2 = TPM **und** Tang
`tang`	Liste von Host-IDs	Pflicht bei `nbde`	Tang-Server; einer davon genügt
`fallback`	`remote-ssh` `console`		Was ein Mensch tut, wenn die Methoden scheitern: SSH (initrd-SSH, FileVault per SSH) oder Konsole/KVM

Fall	`unlock`
Unverschlüsselt	weglassen
TPM, Passphrase an der Konsole als Rückfall	`{"methods": ["tpm"], "fallback": "console"}`
Nur ein Mensch, per SSH (z. B. FileVault)	`{"methods": [], "fallback": "remote-ssh"}`
TPM **oder** Tang	`{"methods": ["tpm", "nbde"], "tang": ["a"]}`
TPM **und** Tang (Platte entsperrt nur im eigenen Netz)	`{"methods": ["tpm", "nbde"], "threshold": 2, "tang": ["a"]}`

Fall	`unlock`
Nur Tang, zwei Server	`{"methods": ["nbde"], "tang": ["a", "b"], "fallback": "console"}`

NBDE ist eine eigene Abhängigkeit: Ein Host, der Tang **braucht** (`nbde` ohne `tpm`, oder `threshold` 2), kommt erst wieder hoch, wenn einer seiner Tang-Server selbst ohne Hilfe läuft. Die Validierung verfolgt diese Ketten. Gibt es einen Host, der nie ohne Mensch hochkäme – zwei Hosts, die gegenseitig ihr einziger Tang-Server sind, oder ein Tang-Server, der selbst an der Passphrase-Abfrage wartet –, ist das ein Fehler. Das abgeleitete `unattendedBoot` sagt je Host, ob er nach Reboot oder Stromausfall von allein wiederkommt.

Nicht geprüft: ob ein Tang-Server wirklich Tang betreibt. Das steht in seiner Config, nicht hier.

Prüfung

`nix flake check` (oder nur der Inventar-Teil: `just inventory-test`) prüft in drei Stufen. Jede Stufe läuft nur, wenn die vorige nichts gefunden hat, damit aus einem Tippfehler keine zwanzig Folgefehler werden. Jeder Fehler nennt den Pfad:

1. Struktur – zuerst die oberste Ebene, allein, dann jeder Eintrag:

- oben nur `$schema`, `owners`, `uplinks`, `sites`, `networks`, `hosts`, alle fünf Abschnitte vorhanden
- gültige IDs und Interface-Namen, keine unbekannten Felder (fängt Tippfehler), Pflichtfelder vorhanden
- Typen, Enums, Mindest-/Höchstwerte, Formate (MAC, IPv4-Adresse und -Präfix, IPv6-Präfix, Link-Local, SSH-Key), keine Duplikate in Listen
- Referenzen zeigen auf einen vorhandenen Eintrag des richtigen Abschnitts

```
hosts.vps.critcality: unknown field (known: availability, criticality, inside, interfaces, kind,
managed, manages, model, note, owner, power, productId, runsOn, serial, site, sshHostKeys, unlock,
vendor)
hosts.nas.interfaces.eno1.mac: "52-54-00-12-34-56" is not a lower-case MAC address like
aa:bb:cc:dd:ee:ff
hosts.app.runsOn: "pve2" is not in hosts
uplinks.fiber-home.asn: expected a whole number, got "AS64496"
```

2. Regeln zwischen Einträgen:

- `runsOn` nur für `vm`/`container` und nur auf einer erlaubten Art; nicht zusammen mit `inside`
- `manages` nur für `kvm`; nichts zeigt auf sich selbst
- `runsOn`/`inside`/`power` ohne Zyklen
- `site` vorhanden oder erbbar; ein ausdrückliches `site` passt zum geerbten
- jede Site hat einen Uplink; kein Uplink, keine Site, kein Owner ohne Verwendung; IPv6-Details eines Uplinks nur mit `ipv6`
- `unlock`: mindestens eine Methode oder ein Rückfall; `threshold` ≤ Zahl der Methoden; `tang` genau dann, wenn `nbde`
- `managed` stimmt mit `flake.nix` überein: Jeder Eintrag aus `nixosConfigurations`/`darwinConfigurations` steht hier als managed, und jeder managed-Eintrag hat eine Konfiguration
- Hardware: `vendor`/`productId`/`serial` nicht bei `vm`/`container`; keine Seriennummer doppelt
- SSH-Keys: Key-Daten passen zum Typ, kein Key bei zwei Geräten
- Interfaces: Netz an der Site des Hosts; `ipv4` braucht ein Netz, `address` je nach Modus, im Präfix, nicht Netz-/Broadcast-Adresse, statisch außerhalb des Pools, je Netz eindeutig; `reserved` braucht `mac` und einen `dhcpServer`; MAC eindeutig und kein Multicast; `eui64` braucht eine MAC und keine private WLAN-MAC
- Netze: Präfix ohne Host-Bits; Pool im Präfix, Start ≤ Ende, nur mit `ipv4` und `dhcpServer`; `dhcpServer` an der Site und mit Interface im Netz; VLAN-ID je Site eindeutig

```
hosts.app.runsOn: a vm cannot run on nas (kind nas; allowed: hypervisor)
hosts: runsOn/inside/power form a cycle through pve, ups1
hosts.laptop.managed: true, but flake.nix has no configuration of that name
hosts.router.interfaces.lan.ipv4.address: 192.0.2.150 is static but inside the DHCP pool of
network lan
networks.lan.vlan: 20 is also used by networks.iot at site home
hosts.router.sshHostKeys[0]: same key as hosts.nas.sshHostKeys[0] - a cloned machine that kept its
host key?
```

3. Entsperr-Ketten – siehe Entsperren:

```
hosts.nas.unlock: never unlocks unattended - none of its Tang servers (vps) comes up without a
human
```

Getrennt davon, als eigener Check, weil er `ssh-to-age` braucht: `checks.inventory-sops` (siehe SSH-Host-Keys).

```
hosts.cuckoosegg: host key gives age1z9rt…, .sops.yaml has age1z8uz… - host key changed without
sops updatekeys, or a wrong key in inventory.json
```

Das Schema in `inventory.schema.json` deckt nur Stufe 1 ab (ohne Referenzen) und ist für den Editor da; maßgeblich ist die Prüfung in `inventory.nix`. Die Fehlermeldungen sind wie der Code englisch.

Abgeleitete Sicht

`lib.inventory.inventory` (auf der Konsole: `just inventory`) ist das geprüfte Inventar mit allem Abgeleiteten. Jeder Eintrag bekommt seine `id`, dazu:

Abschnitt	Abgeleitet
`owners.<id>`	`hosts` – die Hosts, die er administriert
`uplinks.<id>`	`sites` – die Sites an diesem Anschluss
`sites.<id>`	`hosts` – alles dort, geerbte Sites eingeschlossen

Abschnitt	Abgeleitet
`hosts.<id>`	`site` (wirksam, geerbt entlang `runsOn`/`inside`), `virtual`, `uplinks` (seiner Site, primärer zuerst), `guests`, `contains`, `powers`, `managedBy`, `tangClients`, `upstream` (transitiv, nächster zuerst), `unattendedBoot`
`hosts.<id>.interfaces.<name>`	`name`, `vlan` (aus seinem Netz), `ipv6LinkLocal` (bei `eui64` berechnet, bei fester Adresse wie eingetragen, sonst `null`)
`networks.<id>`	`interfaces` – jedes Interface darin (`host`, `interface`, `mac`, `address`, `mode`); `reservations` – was der `dhcpServer` per MAC vergeben muss (`host`, `interface`, `mac`, `address`), nach Adresse sortiert

Neben `inventory`: `lib.inventory.knownHosts` (der Wert für `programs.ssh.knownHosts`) und `lib.inventory.knownHostsFile` (dasselbe als known_hosts-Text).

Optionale Felder sind in der abgeleiteten Sicht immer vorhanden: Standardwerte eingesetzt (`criticality`, `availability`, `unlock.threshold`), sonst `null` bzw. `[]`. Wer sie nutzt, braucht nie ein `or`.

Ist das Inventar ungültig, wirft `inventory` mit **allen** Fehlern auf einmal; `lib.inventory.errors` ist die Liste selbst (leer, wenn gültig).

Beispiele

Die Beispiele sind das Testinventar aus `tests/inventory.nix`, als JSON geschrieben. Es deckt jedes Feld und jede Beziehung ab.

Gemietete VM – virtuell, der Hypervisor des Anbieters steht nicht im Inventar:

```
"vps": { "kind": "vm", "site": "dc", "owner": "lab", "managed": true, "criticality": "core" }
```

VM auf dem Hypervisor anderer, der an einer USV hängt – `site` wird von `pve` geerbt:

```
"pve":  { "kind": "hypervisor", "site": "home", "owner": "club", "managed": false, "power":
        "ups1" },
"ups1": { "kind": "ups", "site": "home", "owner": "lab", "managed": false },
"app":  { "kind": "vm", "runsOn": "pve", "owner": "lab", "managed": true,
          "unlock": { "methods": ["nbde"], "tang": ["nas"] } }
```

Abgeleitet für `app`: `site` = `home`, `upstream` = `["pve", "ups1"]`, `unattendedBoot` = `true` (über die Kette app → nas → vps).

NAS mit TPM und Tang gemeinsam, darin eine PCIe-KVM-Karte:

```
"nas": { "kind": "nas", "vendor": "Example Corp", "model": "Example NAS", "productId": "EX-1000",
         "serial": "NAS0001", "site": "home", "owner": "lab", "managed": true,
         "unlock": { "methods": ["tpm", "nbde"], "threshold": 2, "tang": ["vps"] } },
"kvmcard": { "kind": "kvm", "inside": "nas", "power": "nas", "manages": ["nas"],
             "owner": "lab", "managed": false }
```

Abgeleitet für `nas`: `contains`, `powers` und `managedBy` = `["kvmcard"]`, `tangClients` = `["app"]`.

Laptop – offline ist normal, nur ein Mensch entsperrt:

```
"laptop": { "kind": "laptop", "site": "home", "owner": "lab", "managed": true,
            "availability": "intermittent", "unlock": { "methods": [], "fallback": "console" } }
```

Netze, eine Reservierung und ein Router – der Router vergibt die Adresse von `nas` per MAC; seine eigene ist statisch, außerhalb des Pools:

```
"networks": {
  "lan": { "name": "Home LAN", "site": "home", "ipv4": "192.0.2.0/24", "ipv6": "2001:db8:1::/64",
           "dhcpServer": "router", "dhcpPool": { "start": "192.0.2.100", "end": "192.0.2.199" } },
  "iot": { "name": "IoT", "site": "home", "vlan": 20, "ipv4": "198.51.100.0/24" }
},
"router": { "kind": "router", "site": "home", "owner": "lab", "managed": false,
            "sshHostKeys": ["ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI…"],
            "interfaces": { "lan": { "network": "lan", "ipv4": { "mode": "static", "address": "192
        .0.2.1" } } } }
```

Abgeleitet: `networks.lan.reservations` = `nas/eno1` (192.0.2.10) und `app/ens18` (192.0.2.11); `nas.interfaces.eno1.ipv6LinkLocal` = `fe80::5054:ff:fe12:3456`; known_hosts-Namen für `nas` = `nas`, `nas.peer.example`, `192.0.2.10`.

Site mit Hauptanschluss und LTE-Backup:

```
"uplinks": {
  "fiber-home": { "isp": "Example ISP", "asn": 64496, "medium": "fiber",
                  "bandwidth": { "down": 1000, "up": 500 }, "ipv4": "cgnat", "ipv6": true },
  "lte-backup": { "medium": "lte" }
},
"sites": {
  "home": { "city": "Exampletown", "kind": "residential", "uplinks": ["fiber-home", "lte-
        backup"] }
}
```

Woanders lesen. In Nix (eine weitere Flake-Ausgabe, ein Modul über `specialArgs`):

```
self.lib.inventory.inventory.hosts.app.upstream   # [ "pve" "ups1" ]
```

In OpenTofu – die Rohdaten, ohne abgeleitete Werte und ohne Prüfung (die bleibt bei `nix flake check`):

```
locals {
  inventory = jsondecode(file("${path.module}/../../inventory.json"))
  servers   = [for id, h in local.inventory.hosts : id if h.managed]
}
```

Tests

`tests/inventory.nix` enthält ein vollständiges, erfundenes Inventar (die Beispiele oben) und 72 Fälle. Jeder Fall ändert genau eine Sache und nennt die erwarteten Fehler – exakter Wortlaut und Reihenfolge – oder, bei gültigen Varianten, Werte der abgeleiteten Sicht. Ein Fall schlägt auch fehl, wenn ein kaputtes Inventar `.inventory` nicht werfen lässt.

Gruppe	Deckt ab
gültig	Basisinventar mit allen abgeleiteten Werten; Container auf einer VM; Tang-Schleife durch `threshold` 1 aufgelöst; Flake-Abgleich abgeschaltet
Uplinks	IPv6-Präfixlänge außerhalb des Bereichs, IPv6-Details ohne IPv6
Struktur	Unbekannter/fehlender Top-Level-Schlüssel, Tippfehler im Feldnamen, Enum, Pflichtfeld, Typ, Mindestwert, Referenz ins Leere und in den falschen Abschnitt, ungültige ID, Duplikate

Gruppe	Deckt ab
Regeln	`runsOn` auf Nicht-Virtuellem und auf falscher Art, `runsOn` + `inside`, `manages` außerhalb eines KVM und auf sich selbst, Selbstbezug, Zyklus, fehlende/nicht erbbare/widersprüchliche `site`, Site ohne Uplink, unbenutzte Einträge, Inventar ≠ Flake
Entsperren	Kein Weg zum Entsperren, `threshold` zu hoch oder ohne Methoden, `nbde` ohne `tang` und umgekehrt, eigener Tang-Server, Tang-Schleife, Tang-Server, der auf einen Menschen wartet
Hardware	`vendor`/`serial` bei einer VM, `model` bei einer VM erlaubt, Seriennummer doppelt; abgeleitete Hardware-Felder und Standardwerte
SSH-Keys	Falsches Format, mit Kommentar, Key-Daten eines anderen Typs, derselbe Key bei zwei Hosts; abgeleitete known_hosts-Einträge und -Datei
Interfaces	MAC-Format, Interface-Name, Link-Local-Wert, MAC doppelt, Multicast-MAC, `eui64` aus privater WLAN-MAC und ohne MAC, Netz an anderer Site, `ipv4` ohne Netz, Adresse je Modus, reserviert ohne MAC oder DHCP-Server, Adresse außerhalb/Broadcast/im Pool/ohne Präfix/doppelt; abgeleitetes Link-Local (EUI-64 einer QEMU-MAC, fest, zufällig) und VLAN
Netze	Präfix-Format, VLAN-Bereich und Duplikat je Site (dieselbe ID an anderer Site erlaubt), Host-Bits, Pool außerhalb/rückwärts/ohne Server, DHCP-Server ohne Interface oder an anderer Site; abgeleitete Reservierungen

```
just inventory-test
```

baut `checks.<system>.inventory` (die echte Datei ist gültig), `inventory-tests` (alle Fälle bestehen) `inventory-schema` (Schema aktuell) und `inventory-sops` (Host-Keys ↔ `.sops.yaml`). Bei einem Fehler listet die Meldung jeden fehlgeschlagenen Fall mit „want“ und „got“.

Eine neue Regel bekommt im selben Commit einen Fall: in `base` mit `set`/`del` genau eine Sache ändern, die exakte Meldung eintragen. **Geänderter Wortlaut** einer Meldung lässt die Tests fehlschlagen – Fall und diese Seite zusammen anpassen.

Abläufe

Was	Schritte
Neuer Host / neues Gerät	Eintrag in `inventory.json` (bei einem Flake-Host `"managed": true` – sonst schlägt `nix flake check` ohnehin fehl), `just inventory-collect`, `network`/`ipv4` von Hand, `just inventory-test`
Host-Key geändert (Neuinstallation)	`just inventory-collect --force <host>`, `.sops.yaml` + `sops updatekeys` (siehe Schlüssel rotieren), deployen, `just known-hosts`
Neue DHCP-Reservierung	Interface mit `mac`, `network`, `ipv4.mode: reserved` + `address`; die Liste für den Router: `networks.<id>.reservations` in `just inventory`
Details eines Anschlusses nachtragen	Felder in `uplinks.<id>`, sonst nichts zu tun
Neues Feld oder neue Geräteart	`spec` in `inventory.nix`, dann `just inventory-schema`, ein Testfall in `tests/inventory.nix`, diese Seite (de/en)
Ansehen	`just inventory`

Services

Dienste sind nach Funktion benannt, nicht nach Produkt. Es gibt drei Arten der Erreichbarkeit: direkt öffentlich, öffentlich hinter dem NetBird-Reverse-Proxy mit SSO, oder nur im Mesh.

Name (`.homelab.example`)	App	Host	Erreichbar
`auth`	Pocket ID	zummi	öffentlich (nginx)
`mesh`	NetBird Management/ Dashboard	grammi	öffentlich (Traefik terminiert TLS)
`s3.aldrin`	Garage S3-API	aldrin	öffentlich (nginx)
`rezepte`	Mealie	grammi	Reverse Proxy + SSO
`grist`	Grist	grammi	Reverse Proxy + SSO
`logs`	logchef	grammi	Reverse Proxy + SSO
`metrics`	Grafana	grammi	Reverse Proxy + SSO
`vm` / `vl`	VictoriaMetrics / VictoriaLogs UI	grammi	Reverse Proxy + SSO (über Traefik-Entrypoint `apps`, Port 8090)
`s3-admin`	garage-ui	aldrin	nur Mesh
`dokumente`	Paperless + paperless-gpt (`/paperless-gpt/`)	ursalia	nur Mesh; öffentlicher Zugang über den Proxy in Arbeit
`llm`	Ollama (hinter Caddy)	zorlock	nur Mesh
`doku`	Diese Dokumentation (private Fassung)	aldrin + ursalia	im Mesh nur Admin-Geräte (warum), ein A-Record je Host; von außen über den Reverse Proxy eine Weiterleitung auf die öffentliche Doku

Reverse Proxy: Traefik auf grammi besitzt Port 443. `mesh.homelab.example` terminiert es selbst, alles andere reicht es per TCP-Passthrough mit PROXY-Protokoll v2 an `netbird-proxy` weiter, das TLS und ACME (tls-alpn-01) selbst macht. Login am Proxy ist SSO gegen Pocket ID, begrenzt auf die NetBird-Gruppen der jeweiligen App.

Mesh-only-Namen: Jeder Name ist eine eigene NetBird-DNS-Zone mit einem A-Record auf die Mesh-Adresse (192.0.2.x). Öffentliches DNS kennt sie nicht; das Zertifikat kommt per DNS-01 über deSEC.

Querschnitt auf allen Linux-Hosts: vmagent schickt Metriken, Vector schickt journald- und nginx-Logs an grammi. CrowdSec liest die nginx/Traefik-Logs auf den öffentlichen Hosts. restic-Backups gehen in Garage-Buckets auf aldrin, für Paperless zusätzlich append-only zu BorgBase.

Module

Die 17 Module unter `modules/` tragen die gemeinsame Konfiguration aller Hosts; jede Datei erklärt im Kopfkommentar, warum sie so ist. Die öffentliche Fassung enthält sie anonymisiert unter `modules/` – als Referenz, nicht direkt importierbar: einige lesen `hosts/<host>/*.yaml` und setzen sops-nix voraus. Vier haben eigene Optionen.

Modul	Zweck	Optionen / Besonderheiten
auto-upgrade-git.nix	Nächtliches `nixos-rebuild switch`, committet und pusht die neue `flake.lock` zurück ins Repo	`services.autoUpgradeGit`: `enable`, `flakeDir`, `flakeAttr`, `updateInputs`, `dates`, `allowReboot`, `rebootWindow`, `deployKeySecret`, `gitAuthorEmail`
log-shipping.nix	journald- und nginx-Logs per Vector an VictoriaLogs	`services.logShipping`: `enable`, `victoriaLogsUrl`
metrics-shipping.nix	vmagent scrapt die lokalen Exporter und schreibt per remote_write an VictoriaMetrics	`services.metricsShipping`: `enable`, `remoteWriteUrl`, `exporterPorts`

Modul	Zweck	Optionen / Besonderheiten
deploy.nix	Die Lab-Policy für auto-upgrade-git: GitHub-Deploy-Key, gepinnter Hostkey, Reboot-Fenster 04:00–06:00	getrennt vom Mechanismus, damit der wiederverwendbar bleibt
acme.nix	ACME-Defaults für jeden TLS-Host: DNS-01 gegen deSEC	Token aus `hosts/<host>/desec.yaml` (von Tofu)
netbird.nix	NetBird-Client im Mesh, Registrierung per Setup-Key, lokale Metriken	Setup-Key aus `hosts/<host>/netbird.yaml` (von Tofu)
tailscale.nix	Gemeinsamer Tailscale-Agent	`tailscale serve` bleibt in der Host-Config
ssh.nix	Wer sich von wo anmelden darf: root nur über Mesh oder Tailscale	Schlüssel bleiben pro Host
firewall.nix	Öffentlich nur 22/80/443, Tailnet als vertrauenswürdiges Interface	
crowdsec.nix	CrowdSec-Engine und Firewall-Bouncer, Whitelist für die Lab-Hosts	Umgehungen für Fehler in den nixpkgs-Modulen, im Kopf dokumentiert
networking.nix	ifstate-Gerüst, systemd-resolved mit DNS-over-TLS, keine Fallback-Resolver	Adressen und Routen bleiben in der Host-Config
metrics.nix	Prometheus-Exporter pro Host	Schutz über die Firewall statt über Bind-Adressen
logging.nix	Begrenzt journald, das sonst bis 4 GB wächst	
qemu-guest.nix	QEMU Guest Agent mit gesperrter RPC-Liste: Snapshots ja, Shell und Dateizugriff über den Kanal nein	
nix.nix	Nix-Daemon, Garbage Collection, Store-Optimierung	
docs-site.nix	Die private Doku-Site (Flake-Paket `docs-site`) unter `doku` im Mesh	`services.docsSite`: `enable`, `package`, `fqdn`, `allow` (Standard: das ganze Mesh; im Lab nur die Admin-Geräte aus `admin-peers.json`); auf aldrin und ursalia
base.nix	Locale, Konsole, Zeit und Werkzeuge zum Debuggen	

just-Rezepte

Rezept	Zweck
`just check <host>`	`git pull` + `nixos-rebuild dry-activate`
`just secrets <host>` / `pull-secrets <host>`	Host-Secrets auf dem Host editieren und zurückholen
`just topology`	Topologie-Dashboard neu erzeugen (sonst schlägt `nix flake check` fehl)
`just inventory`	Das geprüfte Inventar mit allem Abgeleiteten anzeigen (Inventar)
`just inventory-test`	`inventory.json` prüfen, die Tests der Validierung laufen lassen, Schema auf Aktualität prüfen
`just inventory-schema`	`inventory.schema.json` nach einer Formatänderung neu erzeugen
`just inventory-collect [--force] [host …]`	SSH-Host-Keys, Interfaces (MAC, Medium, Link-Local) und Hardware (Hersteller, Modell, Produkt-ID, Seriennummer) von den Geräten in `inventory.json` holen – füllt Lücken und meldet Abweichungen; `--force` übernimmt die Werte des Geräts

Rezept	Zweck
`just known-hosts`	`~/.ssh/known_hosts.nixlab` auf dem MacBook aus dem Inventar schreiben
`just netbird-plan` / `netbird-apply`	NetBird-Account ändern; apply schreibt auch `expected-peers.json` (committen, grammi deployen) und `admin-peers.json` (committen, aldrin und ursalia deployen) neu
`just netbird-proxy-certs`	Proxy-Diensten ohne Zertifikat eins holen und das Dashboard nachziehen (NetBird #7118); läuft am Ende von `netbird-apply`
`just netbird-key <host>`	Neuer Setup-Key für einen Host, 24 h Zeit zum Deployen
`just netbird-import`	Verlorenen Tofu-State aus dem Account wiederherstellen
`just netbird-renew-token`	Jährliche Erneuerung des Service-User-Tokens
`just github-key <host>`	Neuer GitHub-Deploy-Key für einen Host (Schlüssel per sops, Registrierung per Tofu)
`just github-plan` / `github-apply`	Repos und Deploy-Keys auf GitHub ändern
`just desec-key <host>` / `desec-verify`	deSEC-Token tauschen bzw. Scopes von jedem Host aus prüfen
`just gc <host>`	Store-Generationen älter als 7 Tage löschen
`just docs`	Diese Doku bauen: private PDFs (de/en) und die anonymisierte öffentliche Fassung (README, PDFs, Site-Quellen, Module) in `public-docs/` (Clone von `nixlab-docs`, gitignored), dort committet und gepusht; GitHub baut daraus die Site
`just docs-serve`	Private Site mit echten Werten lokal bauen und unter `http://127.0.0.1:8000` zeigen

Rezepte, die sops auf dem MacBook brauchen, laufen mit vorangestellter Anmeldung bei 1Password, die den age-Key über `SOPS_AGE_KEY_CMD` bereitstellt:

```
eval $(op signin) && export SOPS_AGE_KEY_CMD='op read "op://<vault>/<item>/password"' && just
        netbird-plan
```

Troubleshooting

- Nach einem `nixos-rebuild` sagt `systemctl --failed` nichts über die Vergangenheit – der Switch räumt Failed-States ab. Stattdessen `journalctl -u <unit>`.
- NetBird down, Tailscale noch da: `NIXLAB_DOMAIN=example-tailnet.ts.net` vor das Rezept setzen.
- Tofu-Rezepte verweigern den Lauf gegen leeren State, damit keine zweite `servers`-Gruppe oder Token-Garnitur entsteht.
- WLAN auf ursalia: `iw dev wlp2s0f0 link` zeigt Band, Signal und PHY-Modus. Das 5-GHz-Netz des Routers liegt auf einem DFS-Kanal mit 300 TU Beacon-Intervall, ein einzelner Scan kann es verpassen; Band, Signal, Raten und PHY-Modus gehen auch an VictoriaMetrics (node_exporter-Collector `wifi` plus `wifi_link_*`), Grafana-Dashboard „Homelab / WLAN“.
- Nach einem Netzwechsel kann NetBird minutenlang an einem toten P2P-Weg festhalten und trotzdem `Connected` zeigen (gesehen auf ursalia, 2026-10-19). Abhilfe: `systemctl restart netbird-mesh` auf dem Host, notfalls über Tailscale.
- Ein Proxy-Dienst hängt im NetBird-Dashboard bei „Issuing certificate…“ oder liefert von außen einen TLS-Fehler („internal error“): `just netbird-proxy-certs` – fragt die Namen ohne Zertifikat von außen ab und startet `netbird-proxy` auf grammi neu (NetBird-Diskussion #7118). Läuft auch am Ende von `just netbird-apply`.

Konventionen

- Pro Thema ein eigener Git-Worktree (`.worktrees/<thema>`), nach Abschluss entfernen und Branch löschen. Tofu-State bleibt dabei immer im Haupt-Checkout.
- Erst planen, dann nach Freigabe umsetzen.
- Subdomains nach Funktion (`rezepte`, nicht `mealie`).
- Neue Webdienste auf grammi nutzen PostgreSQL statt SQLite.
- Dual-Stack, wo immer möglich.
- Kommentare im Code erklären das *Warum*, inklusive der Fehler, die zu einer Entscheidung geführt haben, mit Datum.
- Code-Kommentare, Commit-Messages und Changelogs auf Englisch; die Projektdoku gibt es auf Deutsch und Englisch.
- Die Doku-Site lädt nichts von Dritten: Schriften und glightbox liegen in der Site selbst (`docs/assets.nix`), statt von Google Fonts bzw. unpkg.com zu kommen. Grund: DSGVO und eine private Doku, die ohne Internet funktioniert.

Offene Punkte

☐ NetBird-Mesh auf Dual-Stack (IPv6-Overlay) umstellen
☐ Reverse-Proxy-Stufe 3 abschließen: Paperless öffentlich unter `dokumente` (Uploads, paperless-gpt)
☐ Upstream-PRs im NetBird-Provider (#247 `direct_upstream`/Cluster-Targets, #248 `crowdsec_mode`); danach Proxy-Targets auf grammi auf `localhost` umstellen
☐ Tailscale von den Servern entfernen, sobald nichts mehr darauf angewiesen ist
☐ Grafana-Alerting (Kontaktpunkt fehlt noch) für SMART, fwupd, Paperless, NetBird Soll ≠ known, WLAN von ursalia (nicht verbunden, hängt auf 2,4 GHz, schwaches Signal)
☐ Update-Benachrichtigung für die von Hand gepinnten Inputs (NetBird, Pocket ID)
☐ Root-Passwörter deklarativ über sops
☐ Host-Liste an einer Stelle führen statt in `flake.nix`, den Tofu-Modulen und `justfile`